Repeat Testing with Mock Data

Medicaid Claims Checker · uploading the same test files round after round · October 21, 2026

> **The short version.** In development, Manual Upload has a checkbox, **Test upload: allow re-submission**, ticked by default. While it is ticked, upload the files in `mock_data/` as they are, as often as you like. Untick it only to test the duplicate check itself.

Why a file is refused without the checkbox

The app accepts each claim only once, on purpose. It refuses a file it has already received, and any claim whose content matches a claim already submitted, even in a different file. The refusal names the earlier batch, shows in Batch History, and emails the admin users. This is the production behaviour. “Clear display” does not reset it; it only hides old batches.

The checkbox lets test files skip that check. It appears only when the app runs in development (`config/dev.exs` sets `allow_test_resubmission: true`); production never shows it, and the server ignores the request if it is sent anyway.

Each round

1. Start the app with `./start.sh` and open `http://localhost:4000/`.
2. In Manual Upload, check that **Test upload: allow re-submission** is ticked. It is ticked again each time the page loads.
3. Drag in files from `mock_data/` (up to 100 files, 10 MB each) and click **Process Files**. Each use case has its own folder, for example `mock_data/R_business_rules/R5_exact_thresholds/`; the Frontend Use Cases list says which files go with which case.
4. The batch name ends in “(test re-submission)”. Open it in Batch History and compare each claim with the expected result in the use-case list, `mock_data/README.md`, or `mock_data/manifest.json`.

Leave out `manifest.json`, the `.dsl` rule files (they go on the Rules page) and the claims in `P_rules_page/P2_redundancy_levels/claims/`, which are written for `redundancy_levels.dsl`, not the live rules.

Testing the duplicate refusal itself

1. **Untick** the checkbox.
2. Upload any claim file that has been uploaded before (with or without the checkbox). Expect: “Nothing was evaluated: every claim was already submitted. Refused as already submitted: …”, a red “Refused as already submitted” box on that batch in Batch History, and an alert email to each active admin user.
3. Dismiss the message with its ×, by choosing new files, or with Clear display.

> **Where the alert email goes.** When Phoenix runs with the Brevo mail settings (as `./start.sh` does when they are set), the email goes to the admins’ real inboxes, so each test sends real mail. Without those settings it stays in the app, at `http://localhost:4000/dev/mailbox`.

Checking every file at once

Two scripts check all the mock data without any clicking. Neither adds batches to Batch History, and both can be run any number of times.

Command	What it does
`./test.sh` `./test.sh R5 N7`	Uploads every file through the real Upload and Rules pages (in a test database) and checks the results, for all 59 use cases or only the ones named. Starts and stops its own engine, so a running `./start.sh` is left alone. Pass = no failures in the last line.
`cd phoenix_web` `mix run --no-start ../mock_data/verify.exs …verify.exs --only R5`	Sends each generated file through the same steps as an upload, using the running engine and the dev database, and compares the outcome with `manifest.json`. Pass = the last line reads FAIL 0.

Good to know

Situation	What to do
A file is refused as already submitted	The checkbox was unticked. Tick it and upload again.
The checkbox isn't there	The app isn't running in development. Start it with `./start.sh`; after a change to `config/dev.exs`, restart Phoenix.
X12 error files (X7, E3) or zips with nothing usable (Z4, Z6)	They hold no claims, so they are never recorded and are refused only for what is wrong with them.
A new-provider rule doesn't fire on an X12 file	Provider tenure comes only from JSON claims; use the JSON files for R8.
Z7 (more than 15,000 claims) has no file	It is too big to keep in git. Make one with `elixir mock_data/generate.exs --large 15001`; it is written to `Z_zip_files/Z7_over_15000_claims/`.
Copies with new claim IDs are wanted anyway	`elixir mock_data/retag.exs --tag 3 --out ~/upload_round3 mock_data` writes copies whose claim IDs end in `-3`, with a matching `manifest.json`; check them with `verify.exs --suite ~/upload_round3/mock_data`. Not needed while the checkbox is ticked.